~ Complete Step by Step ~

Git & GitHub

Version Control • Collaboration • Diagrams

Index

1 Version Control & Introduction
2 Installation & Setup
3 Git Basics: Areas & Workflow
4 Commits & History
5 Branches
6 Merging & Conflicts
7 Undoing Changes
8 Remote Repositories
9 GitHub Basics
10 Fork, Pull Request & Collaboration
11 Advanced Git
12 Branching Strategies & Workflows
13 GitHub Features & Actions
14 How Git Works Inside
15 Best Practices & Troubleshooting
16 Practice Labs
17 Interview Q&A
18 Cheat Sheet

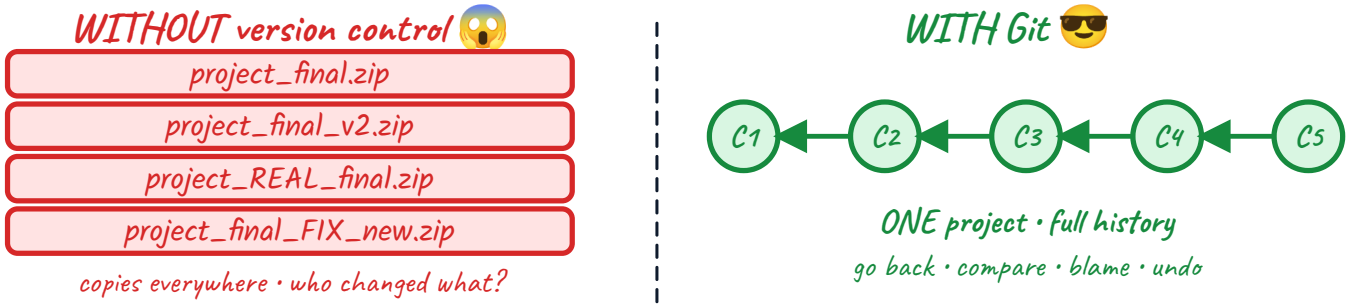
How to use

- Commands are in **blue**, **output / values** in green. Practise **every command** in a test folder – Git is learned by doing!
- Draw the **diagrams** in your notebook: Git is easier when you can *see* commits, branches and remotes.

Notes by asadokz18 ✿

1. Version Control & Introduction

→ **Version control system (VCS)** = software that **records every change** made to files over time, so you can recall any earlier version, compare changes, see who changed what, and work together without overwriting each other.

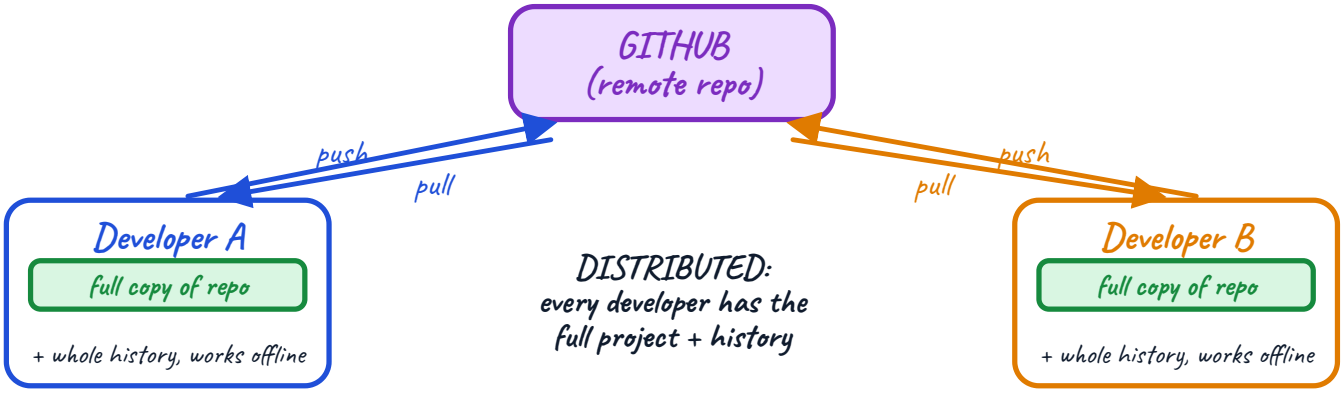


Why use version control?

- → **History:** go back to any earlier state
- → **Backup:** code is safe on GitHub
- → **Teamwork:** many people, same project
- → **Branches:** try new ideas safely
- → **Accountability:** who changed what & why
- → **Review:** check code before merging
- → **Automation:** tests & deploy (CI/CD)
- → **Portfolio:** show your work

Types of version control

Type	How it works	Examples
Local	Versions stored in a database on **one computer**	RCS
Centralized (CVCS)	**One central server**; if it fails, history is lost; needs network	SVN, CVS, Perforce
Distributed (DVCS)	**Everyone has the full repository** + history; works offline; server is just another copy	**Git**, Mercurial



What is Git?

- → Free, open-source **distributed** VCS created by **Linus Torvalds** in **2005** (for the Linux kernel). Fast, branches are cheap, data integrity by **SHA-1** hashes, works with any language. Used by almost every developer.

	Git	GitHub
What	The **tool** (software) on your computer	A **website / service** that hosts Git repositories online
Needs Internet?	No (works offline)	Yes · alternatives: GitLab, Bitbucket, Gitea

Key words: repository (repo) project + history · **commit** saved snapshot · **branch** line of work · **merge** combine · **clone** copy repo · **remote** repo on a server · **origin** default remote name · **HEAD** where you are now · **working tree** your files.

2. Installation & Setup

Install Git

System	How
Windows	Download from **git-scm.com** → run installer (keep defaults) → you get **Git Bash** terminal. Or: ***winget install Git.Git***
macOS	***brew install git*** or ***xcode-select --install***
Linux	***sudo apt install git*** (Ubuntu / Debian) · ***sudo dnf install git*** (Fedora)

Check installation

```
git --version              # git version 2.45.0
git help commit            # manual of a command   (also: git commit --help,  git commit -h)
```

First-time configuration (do once)

Identity & defaults

```
git config --global user.name  "asadokz18"          # shown in every commit
git config --global user.email "you@example.com"    # use the SAME email as your GitHub account
git config --global init.defaultBranch main         # new repos start with "main"
git config --global core.editor "code --wait"       # VS Code as editor
git config --global core.autocrlf input             # line endings (Windows: true, Mac/Linux: input)
git config --global color.ui auto
git config --global alias.st status                 # shortcut:  git st
git config --list                                   # see all settings
```

--system /etc/gitconfig (all users)

--global ~/.gitconfig (YOU, all projects)

--local .git/config (this repo only) – wins!

priority ↑

Priority: local > global > system

```
git config --list
git config --global --edit
```

Create a GitHub account & connect

1. Sign up at **github.com** (free) → verify email → enable **2FA** (two-factor authentication). 2. Choose how to **authenticate** from your computer:

Method	Details
HTTPS + Token (PAT)	URL ***https://github.com/user/repo.git***. Password login is not allowed – use a **Personal Access Token** (Settings → Developer settings → Tokens) or **Git Credential Manager**.
SSH key ✔	URL ***git@github.com:user/repo.git***. Generate a key once; no password typing later.

Generate & add an SSH key

```
ssh-keygen -t ed25519 -C "you@example.com"      # press Enter for defaults (passphrase optional)
eval "$(ssh-agent -s)" && ssh-add ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub                       # copy this PUBLIC key
# GitHub → Settings → SSH and GPG keys → New SSH key → paste
ssh -T git@github.com                           # Hi asadokz18! You've successfully authenticated
```

Never share

your **private** key (***id_ed25519***) or tokens. Only the **.pub** file is shared.

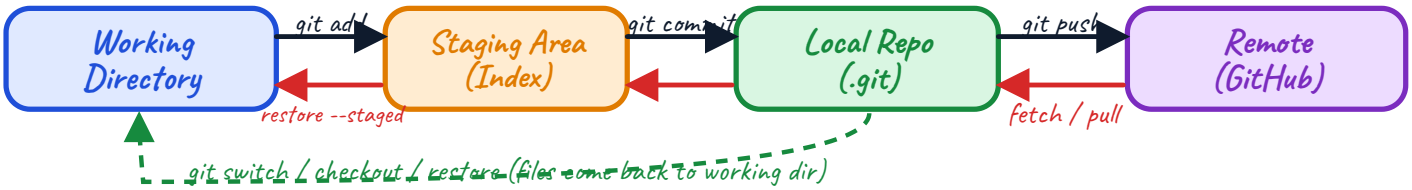
3. Git Basics – Areas & Workflow

➜ A repository (repo) is a project folder that Git tracks. Git stores everything in a hidden folder **.git/** (delete it → no more Git history!).

Start a repository

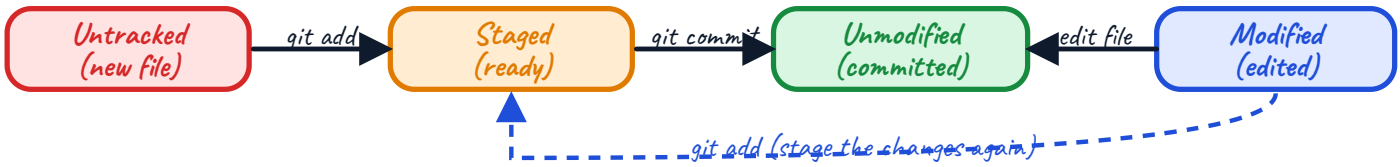
```
mkdir my-project && cd my-project
git init                                  # create a NEW repo here  (creates .git/)
git clone https://github.com/user/repo.git          # OR copy an existing repo
git clone git@github.com:user/repo.git my-folder    # SSH + choose folder name
git clone --depth 1 <url>          # only the latest snapshot (fast)
```

The 3 local areas + remote



Area	Meaning
Working directory	Your actual files & folders – where you edit
Staging area (index)	A "shopping cart": choose **which changes** go into the next commit
Local repository	The **.git** database of all commits (permanent history)
Remote repository	Copy on a server (GitHub) for backup & sharing

Life of a file



git status always tells you which state your files are in

The everyday loop

```
git status                          # what changed?          (green = staged, red = not staged)
git add index.html                  # stage one file         |  git add .  = all  |  git add -p = choose parts
git commit -m "Add homepage"        # save a snapshot        |  git commit -am "msg" = add tracked + commit
git diff                            # unstaged changes       |  git diff --staged = what will be committed
git log --oneline                   # short history
```

Sample git status

```
On branch main
Changes to be committed:  new file:  about.html
Changes not staged:  modified:  style.css
Untracked files:  notes.txt
```

Remember

add = choose · **commit** = save · **push** = share.
Commit **small & often**.

4. Commits & History

A **commit** is a **snapshot** of the whole project at one moment, with a unique **SHA hash** id, author, date, message and a link to its **parent** commit.



ommit points to its PARENT → a chain (history)

Writing good commit messages

- Short summary (≤ 50 characters), **imperative mood**: "Add login form" (not "Added"). Blank line, then optional details (≤ 72 chars/line) explaining **why**.
- **Conventional Commits: *feat:*** new feature · ***fix:*** bug fix · ***docs:*** documentation · ***style:*** formatting · ***refactor:*** · ***test:*** · ***chore:*** build / tools. Example: ***feat(auth): add Google login***

Viewing history

git log options

```
git log                              # full history (q to quit)
git log --oneline --graph --decorate --all     # compact + branch drawing  ← favourite!
git log -5                           # last 5 commits
git log --author="asad"   git log --since="2 weeks ago"   git log --grep="login"
git log -p                           # show changes (patch)      git log --stat   # files changed
git log -- path/file.js              # history of one file       git log -S"function add"  # search code changes
git show a1b2c3d                     # details of one commit     git blame file.js   # who wrote each line
```

Reference	Meaning
HEAD	The commit you are on now (usually the tip of the current branch)
HEAD~1 or HEAD^	Parent of HEAD · **HEAD~3** = 3 commits back
a1b2c3d	Short hash (first 7 characters is enough)
main..feature	Commits in feature that are not in main · ***git diff a1b2..c3d4*** compares two commits

Fix the last commit & manage files

```
git commit --amend -m "Better message"        # change last commit message (only if NOT pushed yet)
git add forgotten.js && git commit --amend --no-edit    # add a forgotten file to the last commit
git rm old.txt          # delete file and stage deletion      git rm --cached secret.env  # stop tracking, keep file
git mv a.txt b.txt      # rename / move (tracked)
```

.gitignore – files Git must not track

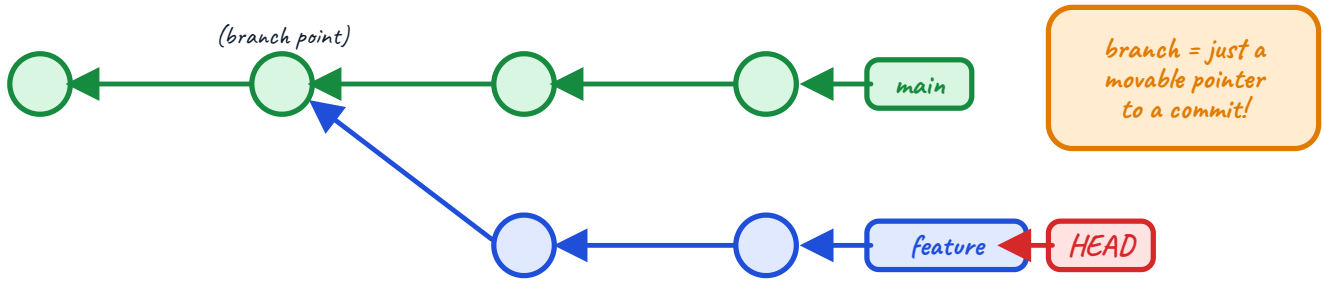
.gitignore

```
node_modules/
.env
dist/
*.log
.DS_Store
!important.log      # ! = exception
/build              # only in root
```

- One pattern per line · * wildcard · **/** end = folder · **#** comment.
- Already tracked? run ***git rm --cached file*** first.
- Templates: **gitignore.io**, github/gitignore. Global: ***~/.gitignore_global***. Empty folder → add ***.gitkeep***.

5. Branches

→ A **branch** is an **independent line of development**. Create one to build a feature or fix a bug **without touching the stable code** (main). Branches are **super cheap** in Git: just a tiny pointer to a commit.



Branch commands

```
git branch                    # list local branches (* = current)      git branch -a  # + remote ones      git branch -vv  # with tracking info
git branch feature/login      # create (stay where you are)
git switch feature/login      # change branch  (modern)       git checkout feature/login   (older way)
git switch -c feature/login   # create AND switch   (git checkout -b feature/login)
git switch -c hotfix abc123    # new branch starting from a specific commit
git branch -m old-name new-name  # rename          git branch -m new-name   # rename current
git branch -d feature/login    # delete (only if merged)   |  git branch -D feature/login  # force delete
git push -u origin feature/login # publish branch to GitHub    |  git push origin --delete feature/login  # delete remote branch
```

Typical branch workflow

1. **git switch main && git pull** - start from the latest code. 2. **git switch -c feature/navbar**. 3. Edit → **git add** → **git commit** (many times). 4. **git push -u origin feature/navbar**. 5. Open a **Pull Request** on GitHub. 6. After merge: delete the branch.

Branch naming conventions

feature/

feature/user-login

bugfix/ hotfix/

bugfix/cart-total
hotfix/payment-crash

release/ docs/ chore/

release/1.2.0
docs/readme

Use lowercase, hyphens, short but meaningful names; include ticket id if you have one: **feature/123-search-bar**.

Compare & switch safely

```
git diff main..feature/login          # differences between two branches
git log main..feature/login --oneline # commits only in feature
git switch other                      # ✘ error if you have uncommitted changes that conflict → commit or  git stash  first
```

Detached HEAD

If you checkout a **commit / tag** (**git checkout a1b2c3d**) HEAD points to a commit, not a branch. Commits made now can get lost. Save them: **git switch -c new-branch**.

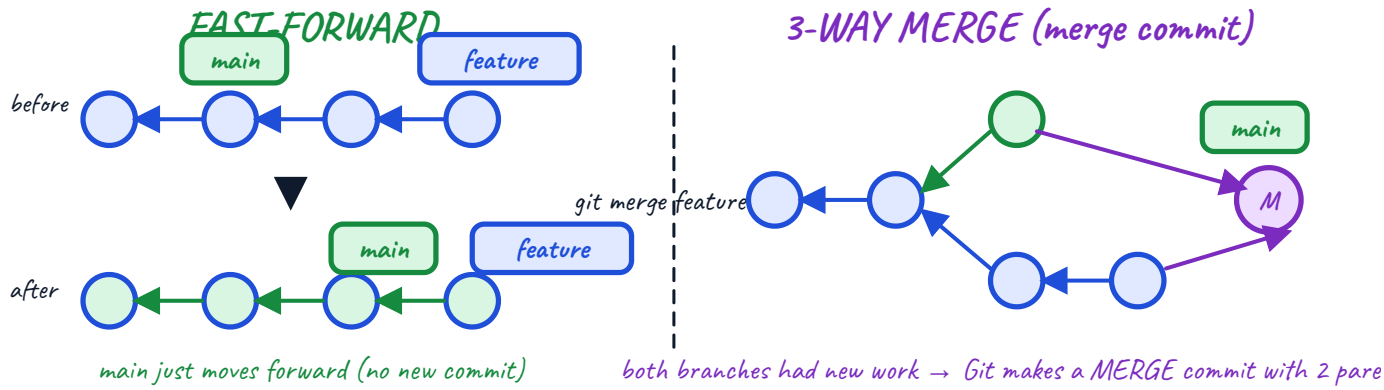
main vs master

Default branch name was **master**, now **main**. Same idea: the stable, deployable branch.

6. Merging & Conflicts

Merge = bring the changes of one branch into another. You stand on the **receiving** branch and merge the other into it.

```
git switch main              # go to the branch that RECEIVES the changes
git merge feature/login      # merge feature into main
git merge --no-ff feature    # always create a merge commit (keeps history of the branch)
git merge --squash feature   # combine all feature commits into ONE staged change (then commit)
git merge --abort            # cancel a merge that has conflicts
```



Merge conflicts

A **conflict** happens when two branches changed the **same lines** (or one deleted a file the other edited). Git stops and asks you to decide.

Conflict markers inside the file

```
<<<<<<< HEAD
<h1>Welcome to my site</h1>          ← your current branch (main) version
=======
<h1>Hello, visitors!</h1>            ← incoming branch (feature) version
>>>>>>> feature/login
```

How to resolve – step by step

1. ***git status*** → see files marked "both modified". 2. Open the file; choose / combine the code and **delete** the markers ***<<<<<<< ======= >>>>>>>***. 3. Test the code. 4. ***git add file***. 5. ***git commit*** (message is pre-filled) ✔. Or ***git merge --abort*** to start over.

- → **Tools:** VS Code shows "Accept Current / Incoming / Both" buttons; ***git mergetool***; GitHub web conflict editor. Choose a side quickly: ***git checkout --ours file*** / ***--theirs file***.
- → **Avoid conflicts:** pull often • keep branches short-lived • small commits • talk to your team • don't reformat whole files • split work by files / areas.

Merge type	When	Result
Fast-forward	main has **no new commits** since the branch started	Pointer moves; linear history
3-way (true) merge	Both branches have new commits	New merge commit with 2 parents

After merging

git branch -d feature/login • ***git push*** • see all merged: ***git branch --merged***.

7. Undoing Changes

Git lets you undo almost anything. First decide: is it **not staged**, **staged**, **committed** or already **pushed**?

I want to...	Command
Discard edits of a file (not staged)	***git restore file.js*** (old: git checkout -- file.js)
Discard ALL unstaged edits	***git restore .*** (cannot be undone!)
Unstage a file (keep edits)	***git restore --staged file.js*** (old: git reset HEAD file.js)
Fix last commit message / add file	***git commit --amend*** (only if not pushed)
Undo last commit, keep changes **staged**	***git reset --soft HEAD~1***
Undo last commit, keep changes **unstaged**	***git reset HEAD~1*** (mixed)
Undo last commit and **delete** changes	***git reset --hard HEAD~1*** ⚠
Undo a commit that is **already pushed**	***git revert a1b2c3d*** (adds a new "opposite" commit - safe)
Restore a file from an old commit	***git restore --source=a1b2c3d file.js***
Delete untracked files	***git clean -n*** (preview) → ***git clean -fd*** (delete)
Go back to remote state completely	***git fetch*** then ***git reset --hard origin/main***

git reset – three modes

Command	HEAD / branch	Staging (index)	Working directory
git reset --soft HEAD~1	moves back 1 commit	kept – still staged	kept – files unchanged
git reset --mixed HEAD~1 (default)	moves back 1 commit	reset – unstaged	kept – files unchanged
git reset --hard HEAD~1	moves back 1 commit	reset	RESET ⚠ changes LOST

soft = undo commit only • mixed = undo commit + staging • hard = undo EVERYTHING (dangerous!)

reset vs revert

reset moves the branch pointer back and **rewrites history** (use on local commits only). **revert** creates a **new commit** that cancels an old one - safe for shared branches.

Golden rule

Never rewrite (***reset / amend / rebase / force push***) commits that others already pulled.

Lost something? git reflog saves you

```
git reflog                    # log of every place HEAD has been (even "deleted" commits)  → a1b2c3d
HEAD@{2}: commit: Add navbar
git reset --hard a1b2c3d      # jump back to that commit      | git switch -c rescue a1b2c3d   # keep it on a new branch
git cherry-pick a1b2c3d       # copy just that commit to the current branch
```

Git keeps unreachable commits ~30-90 days, so **committed work is almost never truly lost**. Uncommitted changes, however, are lost forever after ***restore*** / ***reset --hard*** - commit or ***stash*** first!

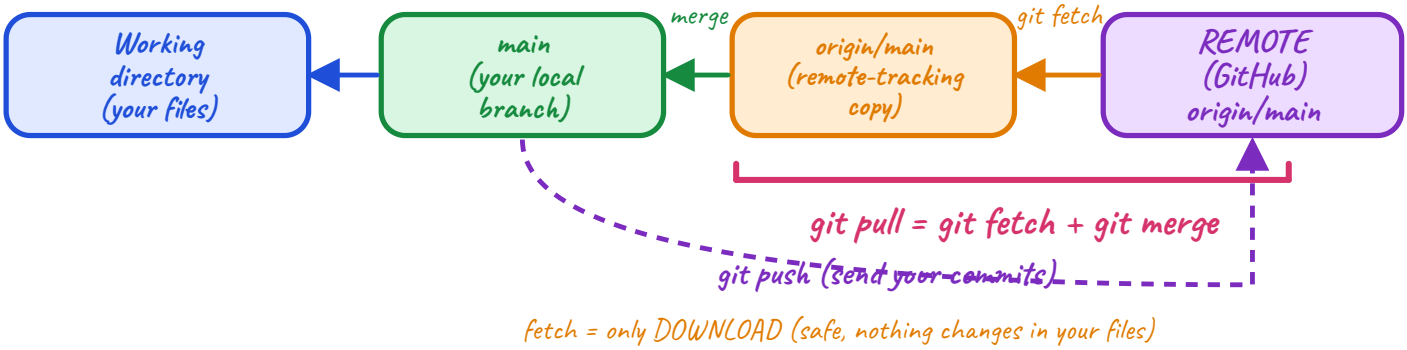
8. Remote Repositories

A **remote** is a version of your repo hosted on a server (GitHub). By default the remote you cloned from is named **origin**.

Remote commands

```
git remote -v                                  # list remotes (fetch / push URLs)
git remote add origin git@github.com:user/repo.git     # connect local repo to GitHub
git remote rename origin github                # rename          git remote remove github    # delete
git remote set-url origin <new-url>            # change URL (e.g. HTTPS → SSH)
git remote add upstream https://github.com/owner/repo.git    # 2nd remote (original project of a fork)
```

push · fetch · pull



Command	What it does
git push origin main	Upload your local commits of **main** to the remote
git push -u origin main	Push + **set upstream** (later just **git push** / **git pull**)
git push --tags	Also upload tags
git push --force-with-lease	Overwrite remote history **safely** (after rebase / amend) – avoid plain **--force**
git fetch (git fetch --all --prune)	Download new commits & update **origin/main** – does **not** change your files
git pull	fetch + merge into current branch. **git pull --rebase** = fetch + rebase (cleaner history)
git branch -vv	Shows which remote branch each local branch tracks

Typical day

```
git switch main && git pull                  # 1. start with the latest code
git switch -c feature/search                 # 2. work on a branch
git add . && git commit -m "feat: add search bar"
git fetch origin && git rebase origin/main   # 3. (optional) update your branch
git push -u origin feature/search            # 4. publish → open Pull Request
```

Authentication reminder

HTTPS

Needs a **Personal Access Token** (or credential manager) instead of password.

SSH ✔

Key pair; URL starts with **git@github.com:**.
Test: **ssh -T git@github.com**

Push rejected?

"*Updates were rejected because the remote contains work you do not have*" → someone pushed first. Run **git pull --rebase**, fix conflicts, push again. Don't force push!

9. GitHub Basics

→ GitHub (2008, owned by Microsoft since 2018) is the world's largest platform to **host Git repositories**, collaborate (Pull Requests, Issues), automate (Actions) and show your work. Over 100 million developers. Your profile = your developer portfolio.

Create a repository on GitHub

1. Click + → **New repository**. 2. Give a **name** (***my-portfolio***) and description. 3. Choose **Public** (everyone sees) or **Private**. 4. Optionally add **README**, **.gitignore** template and **License**. 5. Click **Create repository**.

Push your first project (existing local folder)



Done! Refresh the GitHub page – your code is online 🎉

Repository page tour

Tab	Use
<> Code	Files, branches, commits, green **Code** button (clone URL / download ZIP), README
Issues	Bugs, tasks, feature requests
Pull requests	Review & merge changes
Actions	CI/CD workflows
Projects / Wiki	Kanban boards / documentation
Security / Insights	Vulnerability alerts / contributors, traffic, graphs
Settings	Collaborators, branch protection, Pages, secrets, delete repo

README.md – your project's front page (Markdown)

README.md

```
# Project Title
Short description of what it does.

## Features
- Fast and responsive
- **Bold**, *italic*, `code`

## Install
    npm install && npm start

[Live demo](https://example.com)
![Screenshot](screenshot.png)
```

→ Include: what, why, screenshots, **install / usage**, tech used, contributing, **license**, author.

→ Markdown: ***# Heading***, ***- list***, ***[text](url)***, ***```code```***, tables with |.

Licenses & profile

→ **MIT** – very permissive • **Apache 2.0** – permissive + patents • **GPL** – derivatives must stay open • No license = nobody may legally reuse your code.

→ **Profile README:** create a repo named exactly like your username (***asadokz18/asadokz18***) with a README.md → shown on your profile. Also: **Star** ⭐ repos you like, **Watch** for notifications, **Follow** developers, pin best projects, keep green contribution graph.

10. Fork, Pull Request & Collaboration

Two ways to collaborate

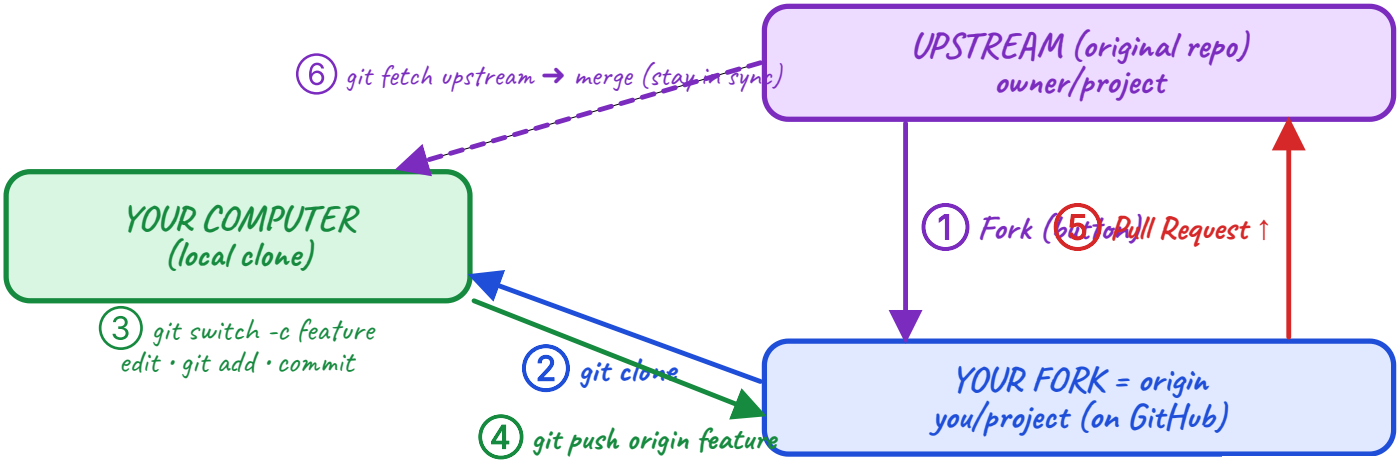
Shared repository

Team members are **collaborators** with write access; everyone pushes **branches** to the same repo and opens PRs.

Fork & Pull model

Used for **open source**: you have no write access, so you **fork** (copy) the repo, work in your copy and send a Pull Request.

Contributing to someone else's project



Commands

```
git clone git@github.com:YOU/project.git && cd project          # ② clone YOUR fork
git remote add upstream https://github.com/OWNER/project.git    # remember the original
git switch -c fix/typo-readme                                   # ③ branch → edit → commit
git push -u origin fix/typo-readme                              # ④ push to your fork → ⑤ click "Compare & pull request"
# keep your fork updated:
git fetch upstream && git switch main && git merge upstream/main && git push origin main     # ⑥
```

Pull Request (PR) lifecycle

1. **Open** PR: base = target branch (main), compare = your branch. Write a clear **title & description** (what, why, screenshots, "Fixes #12" closes the issue). 2. **Automatic checks** run (tests, lint). 3. **Reviewers** comment on lines, **Approve** or **Request changes**. 4. You push more commits to the same branch – PR updates. 5. **Merge** ✔ → delete branch.

Merge button option	Result
Create a merge commit	Keeps all commits + adds a merge commit
Squash and merge	All PR commits → **one** commit on main (clean history, very popular)
Rebase and merge	Commits replayed on main, linear history, no merge commit

Good PR & code review habits

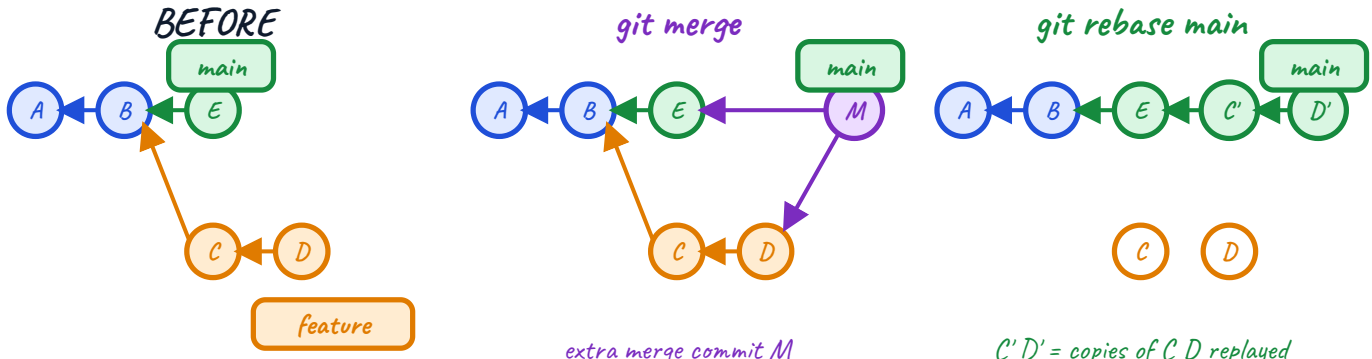
- → Keep PRs **small** (one purpose, <400 lines)
- → Title: *"feat: add dark mode toggle"*
- → Link the issue · add screenshots / how to test
- → Reviewer: be kind & specific, ask questions, suggest changes (**Suggestion** block)
- → Author: respond to every comment, don't take it personally
- → **CODEOWNERS** file auto-requests reviewers

11. Advanced Git

git stash – save unfinished work temporarily

```
git stash                       # hide uncommitted changes (clean working dir)      git stash -u    # include
untracked files
git stash push -m "wip navbar"  # with a name        git stash list      # see stashes
git stash pop                   # re-apply the latest stash and remove it        git stash apply  # re-apply, keep
it
git stash drop stash@{0}        # delete one         git stash clear     # delete all
```

git rebase – a straight history




```
git switch feature && git rebase main       # replay my commits on top of the latest main      (conflict? fix
→ git add → git rebase --continue | --abort)
git rebase -i HEAD~4                        # INTERACTIVE: pick / reword / squash / fixup / drop / reorder the
last 4 commits
git pull --rebase                       # keep local history linear when pulling
```

Golden rule of rebase

Rebase only **your own, unpushed** branches. Rebasing shared history rewrites commit hashes and breaks teammates' work.

11. Advanced Git (continued)

More power tools

Command	Use
`git cherry-pick a1b2c3d`	Copy **one commit** onto the current branch (e.g. hotfix to release)
`git tag v1.0.0` / `git tag -a v1.0.0 -m "msg"`	Lightweight / annotated **tag** = permanent label for a release · `git push origin v1.0.0` or `--tags`
`git bisect start` / `good` / `bad`	**Binary search** through history to find the commit that introduced a bug
`git blame file` · `git log -S"text"`	Who changed each line / when a text appeared
`git diff --stat` · `git shortlog -sn`	Summary of changes · commits per author
`git grep "TODO"`	Search text in tracked files (fast)
`git worktree add ../hotfix main`	Work on 2 branches at once in separate folders
`git submodule add <url>`	Repo inside a repo (shared library). `git clone --recursive`
`git config alias.lg "log --oneline --graph --all"`	Create shortcuts → `git lg`
`git clean -fd` · `git gc` · `git fsck`	Remove untracked · optimise repo · check integrity
`git archive --format=zip HEAD > app.zip`	Export a snapshot without .git

Git hooks – scripts that run automatically

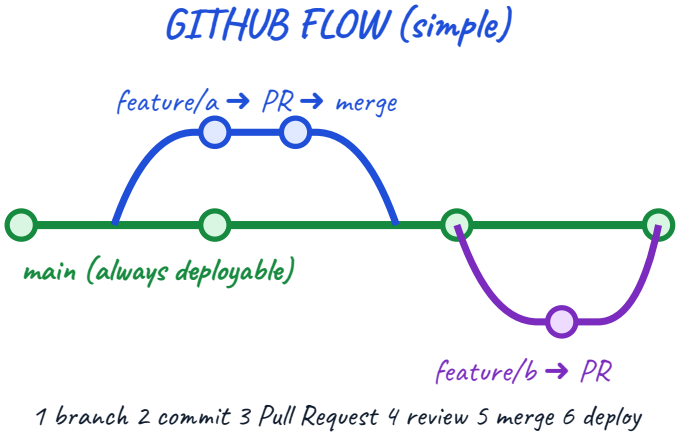
Stored in **.git/hooks/**: **pre-commit** (run lint / tests before commit), **commit-msg** (check message format), **pre-push**. Tools: **Husky** + lint-staged (shared with team), pre-commit framework.

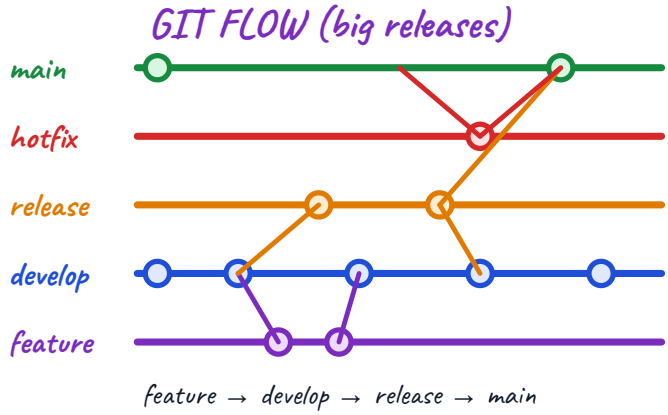
Semantic version tags

v MAJOR . MINOR . PATCH → v2.4.1: **MAJOR** breaking change · **MINOR** new feature (compatible) · **PATCH** bug fix.

12. Branching Strategies & Workflows

A **workflow** is the team's agreement on **how branches are created, reviewed and merged.** Pick the simplest one that fits your project.





Strategy	How it works	Best for
GitHub Flow	**main** is always deployable. Create a short-lived branch → commit → **PR** → review → merge → **deploy immediately**.	Web apps, continuous deployment, small teams
Git Flow	**main** (production) + **develop** (integration) + **feature/*** + **release/*** + **hotfix/***. Releases are prepared on release branches.	Versioned software, scheduled releases (mobile, desktop)
Trunk-Based	Everyone commits to **main (trunk)** in tiny steps or very short branches (<1 day); unfinished work hidden behind **feature flags**.	Mature teams with strong CI & tests
GitLab Flow	main + environment branches (staging, production) or release branches.	Teams that deploy to several environments
Forking Workflow	Contributors work in their own forks, send PRs to upstream.	Open source

Protect your main branch

GitHub → Settings → **Branches** → **Branch protection rules** (or Rulesets):

- ✔ Require a **Pull Request** before merging
- ✔ Require **approvals** (1–2 reviewers)
- ✔ Require **status checks** (tests, lint) to pass
- ✔ Require branches to be **up to date**
- ✔ Require **signed commits** / linear history
- ✔ Restrict who can push, **block force-push** & deletion
- ✔ Require **CODEOWNERS** review
- ✔ Auto-delete merged branches

Releases & versioning

- Tag the release commit: ***git tag -a v1.2.0 -m "Release 1.2.0" && git push origin v1.2.0*** → GitHub **Releases** page: add notes (changelog), attach files, auto-generate release notes. Follow **Semantic Versioning**; keep a **CHANGELOG.md**.
- **Monorepo** (many projects in one repo, e.g. Google) vs **polyrepo** (one repo per project).

Team rules that help

Short-lived branches · small PRs · consistent commit messages · review every change · CI must be green · never commit directly to main · delete merged branches · document the workflow in **CONTRIBUTING.md**.

13. GitHub Features & Actions

Feature	What it does
Issues	Track bugs / tasks. **Labels** (bug, enhancement, good first issue), **assignees**, **milestones**, issue templates; reference with **#12**; close with **Fixes #12** in a PR.
Projects	Kanban / table boards (To do → In progress → Done) linked to issues & PRs.
Actions	CI/CD & automation – run tests, build, deploy on events (push, PR, schedule).
Pages	Free static hosting: **https://user.github.io/repo** (Settings → Pages → choose branch / Actions).
Releases & Packages	Versioned downloads · host npm / Docker packages.
Wiki & Discussions	Documentation pages · community Q&A forum.
Security	**Dependabot** (auto-update vulnerable dependencies), **secret scanning** (finds leaked keys), **code scanning (CodeQL)**, security advisories, SECURITY.md.
Codespaces	Full VS Code dev environment in the browser (cloud).
Copilot	AI pair programmer – code suggestions & chat.
Gists	Share small code snippets / notes (gist.github.com).
Organizations & Teams	Group accounts for companies; roles: owner, member, outside collaborator.

GitHub Actions – automation



If a step fails → ✘ red cross → PR cannot be merged (when branch protection requires checks)

.github/workflows/ci.yml

```yaml
name: CI
on:
  push:         { branches: [main] }
  pull_request: { branches: [main] }
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4          # get the code
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci                        # install
      - run: npm test                      # run tests
      - run: npm run build
        env:
          API_KEY: ${{ secrets.API_KEY }}      # secrets are set in Settings → Secrets
```

→ **Workflow** (yml file) → **jobs** → **steps** · **Event** triggers · **Runner** = machine · **Action** = reusable step · badges for README · schedule with **cron**. Deploy to Pages / Netlify / Vercel / servers after tests pass = **CI/CD**.

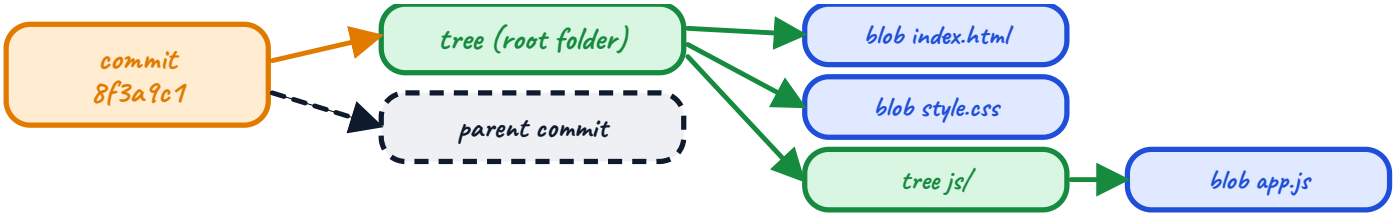
GitHub CLI (gh) & other tools

```
gh auth login          gh repo create my-app --public --source=. --push      gh repo clone user/repo
gh pr create --fill    gh pr list      gh pr checkout 42      gh pr merge 42 --squash      gh issue create
gh run watch
```

Also: GitHub Desktop (GUI), VS Code Source Control panel, GitKraken, **github.dev** (press . on any repo to edit in browser), REST / GraphQL API, webhooks, 2FA & passkeys, Sponsors.

14. How Git Works Inside

Git is a **content-addressable file system**: it stores data as **objects**, each named by a **SHA-1 hash** (40 hex characters) of its content. Same content = same hash; change one byte = new hash. That makes history tamper-evident.



blob = file content · tree = folder · commit = snapshot + message + author + parent · all named by SHA-1 hash

Object	Stores
blob	Contents of one file (no name!)
tree	A directory: list of names → blobs / sub-trees
commit	Pointer to a root tree + parent(s) + author + date + message
tag	Annotated label pointing to a commit

The .git folder

.git/

```
HEAD          # which branch/commit you are on  (ref: refs/heads/main)
config        # repo settings
objects/      # all blobs, trees, commits (compressed)
refs/heads/   # branches = tiny files holding a commit hash
refs/tags/    # tags            refs/remotes/  # origin/* bookmarks
index         # the staging area
hooks/        # automation scripts      logs/  # reflog
```

- A **branch** is just a 41-byte file containing a commit hash - that's why branching is instant. **HEAD** points to the current branch.
- Git stores **snapshots**, not differences (unchanged files are re-used), and packs objects into **packfiles** (delta compression) for space.
- Delete **.git** = project is no longer a repo (files stay). Never edit it by hand.

Plumbing commands

```
git hash-object -w file.txt     # store a blob        git cat-file -p 8f3a9c1   # show object content
git cat-file -t 8f3a9c1         # type                git ls-tree HEAD          # list tree
git rev-parse HEAD                # full hash of HEAD     git count-objects -vH     # repo size
```

Remember

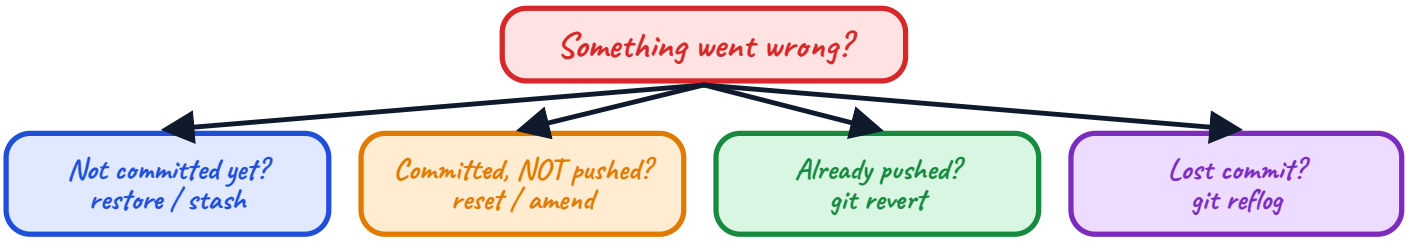
Porcelain = friendly commands (add, commit). Plumbing = low-level commands (hash-object, cat-file).

15. Best Practices & Troubleshooting

- Commit **small & often**, one logical change each
- Write meaningful messages (*feat: …, fix: …*)
- **Pull** before you start & before you push
- Use **branches** for every task
- Never commit directly to **main**
- Keep **.gitignore** up to date
- **Never commit secrets** (.env, keys, passwords)
- Review your diff before committing (***git diff --staged***)
- Don't rewrite shared history
- Use SSH keys + 2FA

Committed a secret?

Rotate (change) the key immediately – deleting the file isn't enough because it stays in history. Then clean with **git filter-repo** or BFG. Large files → **Git LFS**.



Always check first: git status • git log --oneline --graph • git diff

Common errors & fixes

Error / problem	Fix
fatal: not a git repository	Run inside the project folder, or ***git init***
remote origin already exists	***git remote set-url origin <url>***
Permission denied (publickey)	Add SSH key to GitHub; ***ssh -T git@github.com***; check ***ssh-add***
Authentication failed (HTTPS)	Use a Personal Access Token instead of the password
Updates were rejected (non-fast-forward)	***git pull --rebase*** then push
CONFLICT (content)	Edit markers → ***git add*** → ***git commit*** (or ***merge --abort***)
detached HEAD	***git switch -c new-branch*** to keep work, or ***git switch main***
Changes would be overwritten by checkout	Commit or ***git stash*** first
Committed to wrong branch	***git switch right***, ***cherry-pick <hash>***, then reset the wrong one
File still tracked after .gitignore	***git rm --cached file***
Wrong author / email	***git config user.email***; ***commit --amend --reset-author***
Push rejected: file >100MB	Remove it from history; use Git LFS
Line ending warnings (CRLF)	***git config core.autocrlf true*** (Win) / ***input*** (Mac/Linux)

16. Practice Labs

Type every command yourself – Git is learned by doing. Use a throw-away folder.

Lab 1 – Your first repository

```
mkdir git-lab && cd git-lab && git init
echo "# My Lab" > README.md
git status                          # untracked
git add README.md && git commit -m "docs: add README"
git log --oneline
```

Lab 2 – Branch & merge

```
git switch -c feature/hello
echo "Hello" > hello.txt && git add . && git commit -m "feat: add hello"
git switch main && git merge feature/hello
git branch -d feature/hello
```

Lab 3 – Create & resolve a conflict

1. On **main** edit line 1 of README → commit. 2. Create branch ***fix* from the older commit**, edit the same line differently → commit. 3. ***git merge fix*** → CONFLICT. 4. Open file, keep the right text, delete ***<<<<<<< ======= >>>>>>>***. 5. ***git add README.md && git commit***.

Lab 4 – Undo mistakes

```
echo "oops" >> README.md       && git restore README.md          # discard
echo "bad" > bad.txt && git add . && git commit -m "bad"
git reset --soft HEAD~1                                  # undo, keep staged
git commit -m "ok"  &&  git revert HEAD                    # safe undo
git reflog                                             # find anything lost
```

Lab 5 – GitHub & Pull Request

1. Create an empty repo on GitHub. 2. ***git remote add origin <url>*** → ***git push -u origin main***. 3. Create branch ***feature/about***, commit, push. 4. Open a **Pull Request**, review your own diff, **Squash and merge**. 5. ***git switch main && git pull***.

Lab 6 – Mini projects

- **Fork** any open-source repo, fix a typo, send a PR.
- Create your **profile README** (repo = your username).
- Publish a website with **GitHub Pages**.
- Add a **CI workflow** that runs tests on each push.

17. Interview Questions & Answers

1. **What is Git?** A free, open-source **distributed version control system** (by Linus Torvalds, 2005) that tracks changes and lets many people collaborate.
2. **Git vs GitHub?** Git = the tool on your computer. GitHub = a cloud platform hosting Git repos with PRs, issues, Actions. (Alternatives: GitLab, Bitbucket.)
3. **What is a repository?** A project folder with its full history stored in ***.git***.
4. **Working directory, staging area, repository?** Your files → files chosen for next commit (index) → permanent history.
5. **Explain *git add* and *git commit*.** add stages changes; commit saves a snapshot of staged changes with a message.
6. **Clone vs fork?** Clone copies a repo to your computer. Fork copies a repo to your GitHub account.
7. **Fetch vs pull?** fetch only downloads; pull = fetch + merge (or rebase).
8. **Merge vs rebase?** Merge keeps history and adds a merge commit; rebase replays commits for a linear history (don't rebase shared branches).
9. **Reset vs revert?** reset moves the branch back (rewrites history); revert adds a new commit that undoes one (safe when pushed).
10. **soft, mixed, hard reset?** Keep staged / keep unstaged / discard everything.
11. **What is HEAD?** A pointer to the current branch / commit you're on. **Detached HEAD** = pointing directly at a commit, not a branch.
12. **What is a merge conflict & how to solve?** Same lines changed differently. Edit markers, ***git add***, commit.
13. **What is a fast-forward merge?** Target branch has no new commits, so Git just moves the pointer - no merge commit.
14. **What does *git stash* do?** Temporarily shelves uncommitted changes; ***stash pop*** brings them back.
15. **What is cherry-pick?** Apply one specific commit to the current branch.
16. **What is .gitignore?** A file listing patterns Git should not track (node_modules, .env).

17. Interview Q&A (continued)

1. **What is a Pull Request?** A request to merge your branch into another, enabling review, discussion and automated checks.
2. **Squash merge?** Combines all PR commits into one commit on the target branch.
3. **How do you undo a pushed commit?** ***git revert <hash>*** and push. (Avoid force push on shared branches.)
4. **How to recover a deleted branch / lost commit?** ***git reflog***, find the hash, ***git switch -c name hash***.
5. **What is *--force-with-lease*?** A safer force push that fails if someone else pushed in the meantime.
6. **What is origin / upstream?** origin = your default remote; upstream = the original repo of a fork (or the tracked remote branch).
7. **What is a tag?** A fixed label (e.g. v1.0.0) for a release commit.
8. **What is Git Flow vs GitHub Flow?** Git Flow: main, develop, feature, release, hotfix. GitHub Flow: main + short feature branches + PRs.
9. **What is *git bisect*?** Binary search through commits to find the one that introduced a bug.
10. **What is a bare repository?** A repo without a working directory (only .git data), used as a central server repo.
11. **How do you keep a fork in sync?** Add ***upstream***, ***fetch upstream***, merge / rebase into main, push.
12. **How does Git store data?** As objects (blob, tree, commit) identified by SHA-1 hashes; snapshots, not diffs.
13. **What are Git hooks?** Scripts triggered by events (pre-commit, pre-push) in ***.git/hooks***.
14. **What is CI/CD with GitHub Actions?** Workflows in ***.github/workflows*** that test (CI) and deploy (CD) automatically on push / PR.
15. **How to remove a file from Git but keep it locally?** ***git rm --cached file***.
16. **How to see who changed a line?** ***git blame file***.
17. **How to squash the last 3 commits?** ***git rebase -i HEAD~3*** → mark squash / fixup.
18. **What is Dependabot / branch protection?** Auto dependency updates / rules requiring review & passing checks before merge

Interview tip

Always explain with a tiny example: "I branch, commit, open a PR, a teammate reviews, CI passes, I squash-merge and delete the branch."

18. Git Cheat Sheet

Setup

```
git config --global user.name "N"
git config --global user.email "e"
git init  |  git clone <url>
```

Snapshot

```
git status  |  git diff [--staged]
git add . | -p   git commit -m ""
git commit --amend
git rm f | git mv a b
```

Branch & merge

```
git branch [-a] | -d x
git switch x | -c x
git merge x | --abort
git rebase main | -i HEAD~3
git cherry-pick <h>
```

Undo

```
git restore f | --staged f
git reset --soft|--hard HEAD~1
git revert <h>   git reflog
git clean -fd    git stash [pop]
```

Remote

```
git remote -v | add origin <url>
git fetch | pull [--rebase]
git push [-u] origin x
git push --force-with-lease
git push --tags
```

Inspect

```
git log --oneline --graph --all
git log -p | --stat | --author=
git show <h>   git blame f
git branch -vv   git bisect
```

Tags & release

```
git tag -a v1.0.0 -m "msg"
git push origin v1.0.0
```

GitHub CLI

```
gh auth login | repo create
gh pr create --fill | merge
gh issue create | gh run watch
```

The daily loop

pull → branch → edit → add → commit → push → Pull Request → review → merge → delete branch

✨ Keep committing - practice makes you a Git master! ✨ — asadokz18